

CSE 344 – Systems Programming

Homework #2

Create Two Different Processes and Implement IPC Communication with Daemon Process

Sümeyye Zeynep Gürbüz

200104004039

Objective: Implement a communication protocol between two processes using IPC and introduce a daemon process to handle background operations.

Remarks:

- Instead of perror() methods, I have used 'stderr' with fprintf.
- To compile & run the project:
 - make compile
 - make

Default Definitions:

- _POSIX_SOURCE // enabling POSIX-compliant functions and constants
- SA_RESTART = 0 // disabling auto-restart of interrupted syscalls
- FIFO1 = "MY_FIFO_1" // path to FIFO 1
- FIFO2 = "MY_FIFO_2" // path to FIFO 2
- FILE_MODE = 0666 // permissions for file operations
- SLEEP = 10
- MAX_ARGS = 3
- MAX_CHILDREN = 2
- OPERATION = "find_largest" // operation to be done with FIFOs

Global Variables:

- **chld_counter**: volatile sig_atomic_t // to count the killed children number
- **log_fd**: int // to use log file globally

Functions:

- **int read_with_timeout(int fd, void *buf, size_t count, int timeout_secs)**
 - Attempts to read from a file descriptor (fd) with a timeout. If no data is available within timeout_secs, it returns an ETIMEDOUT error.
 - Prevents the child from waiting forever for data that might never come. If a process (like a faulty child) never writes to a FIFO, this ensures the program doesn't hang indefinitely.
- **void _log(const int fd, const char* msg)**
 - Writes a log message (with a timestamp) to the specified file descriptor. Used for logging events to stdout, stderr, or the log file.
- **void sigchld_handler(int signo)**
 - Handles SIGCHLD signal when a child process exits.
 - Uses waitpid() to determine child status.
 - If a child exits normally and successfully, increments child_counter.
- **void sig_handler(int signo)**
 - Handles various signals:
 - SIGUSR1, SIGHUP: Just logs an event.
 - SIGTERM: Cleans up, logs, and exits safely.
- **void daemonize()**
 - Turns the parent process into a daemon:
 - Detaches from terminal.
 - Forks twice.
 - Closes all open file descriptors.
 - Redirects stdin to /dev/null, and stdout/stderr to log_fd.
- **void close_fifos()**
 - Removes the named FIFO files using unlink.

Main Flow:

- Startup
 - Opens log.txt.
 - Checks argument count.
 - Sets up signal handlers.
- Child Process 1 (pid1)
 - Sleeps for 10 seconds.
 - Reads integers from FIFO_1.
 - Calculates the larger number.
 - Writes the result to FIFO_2.

- Child Process 2 (pid2)
 - Sleeps for 10 seconds.
 - Reads a string command and result from FIFO_2 using `read_with_timeout`.
 - Logs both.
- Parent Process
 - Creates FIFO files (`mkfifo`).
 - Writes the string command to FIFO_2.
 - Writes the integers to FIFO_1.
 - Calls `daemonize()` to go background.
 - Waits for up to 30 seconds for children to finish, checking every 2 seconds.
- If children haven't finished after 30 seconds, it sends `SIGTERM` to both.

Screenshots:

\$ *a.out* 202 158

```

C 200104004039.c  log.txt  makefile
log.txt
139 [2025-04-08 13:45:28]
140 Proceeding...
141
142 [2025-04-08 13:45:30]
143 Proceeding...
144
145 [2025-04-08 13:45:32]
146 Proceeding...
147
148 [2025-04-08 13:45:34]
149 Proceeding...
150
151 [2025-04-08 13:45:36]
152 Proceeding...
153
154 [2025-04-08 13:45:38]
155 Proceeding...
156
157 [2025-04-08 13:45:40]
158 Proceeding...
159
160 [2025-04-08 13:45:40] - Timeout reached, terminating
161 [2025-04-08 13:45:40] --> Received SIGTERM. Exiting..
162 [2025-04-08 13:45:40] --> FIFOs have been unlinked

[2025-04-08 13:44:59] - HI, this is parent talking!
[2025-04-08 13:44:59] - FIFO 1 is created.
[2025-04-08 13:44:59] - FIFO 2 is created.
[2025-04-08 13:45:09] - Greetings from the 1st child!
[2025-04-08 13:45:09] - Hold on a second...
[2025-04-08 13:45:09] - Greetings from the 2nd child!
[2025-04-08 13:45:09] - Hold on a second...
[2025-04-08 13:45:09] - Reading FIFO 2...
[2025-04-08 13:45:09] - Command is written to FIFO 2.
[2025-04-08 13:45:09] - Numbers are written to FIFO 1.
[2025-04-08 13:45:09] - Parent is daemonizing...
[2025-04-08 13:45:09] - Read numbers: koixos@debian12: ~/workspace/hw02$ [2025-04-08 13:45:09] 202 [2025-04-08 13:45:09] 158 [2025-04-08 13:45:09]
- The largest number is found.
[2025-04-08 13:45:09] - The largest number is written to FIFO 2.
[2025-04-08 13:45:09] - First child is successfully completed!
[2025-04-08 13:45:11] - Command and the result, respectively: find_largest - 202
[2025-04-08 13:45:11] - 2nd child is successfully completed!

koixos@debian12: ~/workspace/hw02$
koixos@debian12: ~/workspace/hw02$

```